Facultad de Ingeniería

“Desarrollo de un Gestor Académico Virtual basado en IA Generativa para consultas de información en una app universitaria”

Avance de Proyecto Final I

Pruebas de Software (32852)

DOCENTE

Christian Janderson Quiroz Angulo

INTEGRANTES

Diego Fernando Napa Alvarez **(U22234939)**

Lorena Kim Negrillo **(U22229186)**

Pablo Adrian Boza Torres **(U22239512)**

José Daniel Medina García **(U22232430)**

Percy Alonzo Chabria Loayza **(U20217294)**

Jacknylls Ederly Flores Mamani **(U20306493)**

Lima, Perú - 2026

1. Introducción **3**
1.1. Contexto 3
1.2. Problema identificado 3
1.3. Propuesta de solución 3
1.4. Antecedentes 4
1.4.1. Asistente virtual educativo basado en LLM y RAG 4
1.4.2. Asistente institucional basado en RAG 4
1.4.3. Tutor con RAG aplicado a educación superior 5
1.5. Objetivo general 5
1.6. Objetivos específicos 5
1.7. Alcance 6
1.8. Metodología 6
2. Planificación **6**
2.1. Roles del equipo 6
2.2. Stack Tecnológico 8
2.3. Cronograma 9
3. Análisis **10**
3.1. Requerimientos funcionales 10
3.2. Requerimientos no funcionales 11
3.3. Riesgos 12
3.4. Plan de pruebas 15
3.5. Historias de usuario 16
3.6. Matriz de trazabilidad HU x RF/RNF 21
4. Diseño **22**
4.1. Diagrama de clases UML 22
4.2. Modelo Entidad Relación 23
4.3. Diagrama de despliegue 23
4.4. Diagramas de secuencia 24
4.4.1. Diagrama - Flujo principal (consulta exitosa) 24
4.4.2. Diagrama - Flujo alterno (información insuficiente) 24
4.4.3. Diagrama - Flujo alterno (error de comunicación) 25
4.5. Mockup inicial 26
5. Conclusiones **26**
6. Referencias **27**

1. Introducción

1.1. Contexto

Actualmente, las universidades gestionan grandes cantidades de información académica e institucional, como reglamentos, procedimientos, cursos, sílabos y otros documentos. Esta cantidad de información puede dificultar que estudiantes y docentes encuentren rápidamente el contenido que necesitan.

1.2. Problema identificado

El principal problema identificado es la dificultad para acceder de manera rápida y eficiente a información universitaria confiable debido al volumen y diversidad de documentos disponibles. Los usuarios necesitan localizar información específica dentro de diferentes contenidos académicos e institucionales, lo que puede hacer que el proceso de consulta sea poco práctico.

1.3. Propuesta de solución

Se propone desarrollar un Gestor Académico Virtual basado en IA Generativa y RAG que permita realizar consultas inteligentes sobre información universitaria.

El sistema RAG será desarrollado como un servicio independiente, encargado de procesar los documentos, generar embeddings, recuperar la información relevante y generar respuestas mediante IA. Este sistema se comunicará mediante una API

con un módulo de chat incorporado en Moodle, que será utilizado como interfaz de interacción con los usuarios.

De esta manera, Moodle funcionará como el medio de acceso al asistente, mientras que el sistema RAG constituirá el componente principal de la solución, concentrando el procesamiento y generación de las respuestas. El proyecto no contempla desarrollar nuevamente las funcionalidades académicas o administrativas de Moodle.

1.4. Antecedentes

1.4.1. Asistente virtual educativo basado en LLM y RAG

Khan, Khan y Ali (2025) desarrollaron un asistente virtual educativo basado en modelos de lenguaje y RAG para atender consultas relacionadas con una universidad. El sistema utilizó información obtenida de sitios web universitarios, la procesó y recuperó la información relevante antes de generar las respuestas. El estudio evaluó consultas sobre temas como admisión, selección de cursos e instalaciones universitarias.

1.4.2. Asistente institucional basado en RAG

Kuratomi et al. (2025) desarrollaron un asistente virtual basado en RAG para la Universidad de São Paulo. La solución estuvo compuesta por un módulo de recuperación y otro de generación, evaluando diferentes modelos y configuraciones. El estudio mostró que proporcionar al modelo

fragmentos de documentos relevantes permitió mejorar la precisión de las respuestas frente a utilizar el modelo sin contexto recuperado.

1.4.3. Tutor con RAG aplicado a educación superior

Németh et al. (2025) realizaron un estudio piloto sobre un tutor de inteligencia artificial basado en RAG aplicado en cuatro cursos de educación superior. El tutor utilizaba los materiales educativos proporcionados para responder las consultas de los estudiantes. El estudio analizó las conversaciones y reportó que RAG ayudó a reducir las respuestas incorrectas en temas suficientemente explicados dentro del material utilizado como fuente.

1.5. Objetivo general

Desarrollar y validar un sistema RAG con IA Generativa que permita consultar información universitaria mediante lenguaje natural y genere respuestas relevantes y contextualizadas.

1.6. Objetivos específicos

- Implementar el sistema RAG para procesar y recuperar información universitaria.
- Desarrollar la API que conecte el RAG con el chat de Moodle.
- Realizar pruebas unitarias, de integración y funcionales.

- Evaluar el sistema mediante pruebas de rendimiento, seguridad, usabilidad y regresión.
- Registrar y analizar los resultados de las pruebas para verificar la calidad del sistema.

1.7. Alcance

El alcance incluye el procesamiento de documentos, recuperación de información, generación de respuestas y aplicación de pruebas de calidad. No se contempla modificar ni desarrollar las funcionalidades académicas propias de Moodle.

1.8. Metodología

El proyecto utilizará la metodología Scrum para organizar y gestionar el desarrollo del sistema RAG. El trabajo se realizará de forma iterativa e incremental, mediante 6 Sprints de 2 semanas, permitiendo realizar un seguimiento continuo del desarrollo, las pruebas y la calidad del producto.

2. Planificación

2.1. Roles del equipo

Integrante	Rol	Responsabilidades
Jacknylls Flores	Product Owner	Define las necesidades del producto,

		prioriza los requerimientos y gestiona el backlog del proyecto.
José Medina	Scrum Master / QA Lead	Coordina la metodología Scrum, da seguimiento a los Sprints y supervisa las actividades de pruebas y aseguramiento de calidad.
Lorena Negrillo	Backend / API Developer	Desarrolla la lógica del sistema y la API encargada de comunicar la aplicación universitaria con el sistema RAG.
Diego Napa	AI / RAG Engineer	Desarrolla el sistema RAG, incluyendo procesamiento de documentos, embeddings, recuperación de información y generación de respuestas mediante IA.
Percy Chabria	Frontend Developer / Moodle	Desarrolla e integra el módulo de chat en Moodle para permitir la interacción de los usuarios con el sistema RAG.
Pablo Boza	DevOps / Cloud & Infrastructure Engineer	Gestiona los entornos, infraestructura, despliegue y configuración de los servicios necesarios para el

		funcionamiento del sistema.

2.2. Stack Tecnológico

Componente	Tecnología
Lenguaje del sistema RAG	Python
Framework Backend / API	FastAPI
Modelos de IA / LLM	AWS Amazon Bedrock
Integración con Moodle	API REST
Interfaz de usuario	Moodle + módulo de chat
Sistema RAG	Python + FastAPI
Control de versiones	Git + GitHub
Entorno del código desplegado	Docker

2.3. Cronograma

CRONOGRAMA DEL PROYECTO													
N°	Actividades	Semanas											Resultado esperado
		7	8	9	10	11	12	13	14	15	16	17	18	
1	Sprint 1 — Definición y preparación del sistema RAG													Prototipo funcional del RAG
2	Sprint 2 — Recuperación e integración con Moodle													RAG integrado con Moodle
3	Sprint 3 — Pruebas de rendimiento y seguridad													Resultados de rendimiento y seguridad
4	Sprint 4 — Usabilidad y mejoras													Versión mejorada y usabilidad documentada
5	Sprint 5 — Pruebas de regresión y automatización													Suite de regresión y versión estable
6	Sprint 6 — Validación final y cierre													Sistema validado y listo para entrega

3. Análisis

3.1. Requerimientos funcionales

ID	Requerimiento funcional	Descripción	Prioridad	Criterio de aceptación
RF-01	Ingresar consultas en lenguaje cotidiano	El sistema deberá permitir que el usuario ingrese consultas en lenguaje cotidiano mediante el módulo de chat de Moodle.	Alta	El usuario puede escribir una consulta y enviarla desde el chat sin necesidad de utilizar comandos especiales.
RF-02	Enviar consultas al sistema RAG	El módulo de chat de Moodle deberá enviar la consulta ingresada al sistema RAG mediante una API REST.	Alta	La consulta enviada desde Moodle llega correctamente al servicio RAG y es procesada.
RF-03	Recibir y procesar consultas	El sistema RAG deberá recibir las consultas provenientes de Moodle y procesarlas para iniciar el flujo de recuperación de información.	Alta	El RAG recibe correctamente una consulta válida y genera una solicitud de procesamiento.
RF-04	Procesar la base de conocimiento	El sistema RAG deberá procesar los documentos definidos como fuente de conocimiento para permitir su posterior recuperación.	Alta	Los documentos definidos pueden ser procesados y preparados para realizar consultas.
RF-05	Generar embeddings	El sistema deberá generar representaciones vectoriales de la información procesada para facilitar la búsqueda semántica.	Alta	La información procesada genera embeddings almacenables y utilizables por el mecanismo de recuperación.
RF-06	Recuperar información relevante	El sistema RAG deberá recuperar los fragmentos de información más relevantes de acuerdo con la consulta realizada por el usuario.	Alta	Ante una consulta, el sistema recupera información relacionada con el contenido consultado.
RF-07	Generar respuestas mediante IA	El sistema deberá utilizar un modelo de lenguaje para generar una respuesta basada en la información recuperada por el RAG.	Alta	La respuesta generada utiliza el contexto recuperado y responde a la consulta realizada.
RF-08	Entregar la respuesta al chat	El sistema RAG deberá devolver la respuesta generada al módulo de chat de Moodle mediante la API.	Alta	La respuesta generada llega correctamente al módulo de chat.
RF-09	Mostrar la respuesta al usuario	El módulo de chat deberá mostrar al usuario la respuesta recibida desde el sistema RAG.	Alta	El usuario puede visualizar la respuesta generada dentro del chat.
RF-10	Gestionar consultas sin información suficiente	El sistema deberá informar al usuario cuando la información disponible en la base de conocimiento no sea suficiente para responder una consulta.	Media	Ante una consulta sin información relevante, el sistema muestra un mensaje indicando que no se encontró información suficiente.
RF-11	Gestionar errores de comunicación	El sistema deberá informar cuando ocurra un error durante la comunicación entre Moodle, la API y el sistema RAG.	Media	Si la comunicación falla, el usuario recibe un mensaje de error comprensible en lugar de una respuesta incompleta.

3.2. Requerimientos no funcionales

ID	Requerimiento no funcional	Descripción	Métrica / criterio	Prioridad
RNF-01	Rendimiento	El sistema deberá responder las consultas en un tiempo adecuado para permitir una interacción fluida mediante el chat.	El 95% de las consultas deberá recibir respuesta en un máximo de 10 segundos bajo la carga definida para las pruebas.	Alta
RNF-02	Disponibilidad	El servicio RAG y la API deberán permanecer disponibles durante las pruebas y demostraciones del proyecto.	Disponibilidad mínima de 95% durante el periodo de pruebas planificado.	Alta
RNF-03	Seguridad de la comunicación	La comunicación entre Moodle y el sistema RAG deberá realizarse de forma segura.	Las comunicaciones deberán utilizar HTTPS y los datos sensibles de configuración no deberán estar almacenados directamente en el código fuente.	Alta
RNF-04	Seguridad de acceso a la API	La API deberá validar las solicitudes recibidas antes de procesarlas.	Las solicitudes inválidas o no autorizadas deberán ser rechazadas y generar una respuesta controlada.	Alta
RNF-05	Usabilidad	El módulo de chat deberá permitir realizar consultas de manera sencilla e intuitiva.	Un usuario deberá poder realizar una consulta sin requerir instrucciones técnicas adicionales.	Alta
RNF-06	Calidad de las respuestas	Las respuestas generadas deberán ser relevantes y estar basadas en la información recuperada desde la base de conocimiento.	Al menos el 80% de las consultas del conjunto de evaluación deberá obtener una respuesta considerada relevante según los criterios definidos para las pruebas.	Alta
RNF-07	Escalabilidad	El sistema deberá soportar múltiples consultas simultáneas sin afectar significativamente su funcionamiento.	El sistema deberá soportar al menos 10 consultas simultáneas durante las pruebas de carga sin presentar fallos críticos.	Media
RNF-08	Compatibilidad de integración	La comunicación entre Moodle y el RAG deberá mantener un formato de intercambio consistente.	La API deberá intercambiar la información mediante REST y utilizar un formato estructurado como JSON.	Alta
RNF-09	Mantenibilidad	El código del sistema deberá estar organizado de forma que facilite su mantenimiento y modificación.	El código deberá mantenerse modular, versionado mediante Git y acompañado de documentación técnica básica.	Media
RNF-10	Registro y monitoreo	El sistema deberá permitir identificar eventos importantes relacionados con las solicitudes y respuestas del servicio.	Deberán registrarse como mínimo fecha y hora, estado de la solicitud y tiempo de respuesta, sin exponer información sensible.	Media
RNF-11	Tolerancia a errores	El sistema deberá manejar errores de procesamiento o comunicación sin provocar la interrupción completa del servicio.	Los errores deberán generar respuestas controladas y mensajes comprensibles para el usuario.	Alta
RNF-12	Testabilidad	El sistema deberá permitir la ejecución de pruebas para verificar su funcionamiento y calidad.	Los requisitos implementados deberán poder validarse mediante pruebas unitarias, de integración, funcionales y de rendimiento.	Alta

3.3. Riesgos

1. IDENTIFICACION DE RIESGOS						
ID	**Riesgo**	**Descripcion**	**Probabilidad**	**Impacto**	**Nivel**	**Responsable**
R01	Fallo en el servicio de IA (Amazon Bedrock)	Caida o indisponibilidad del servicio de IA en la nube que impide generar respuestas	Media	Alto	Alto	DevOps (Pablo)
R02	Respuestas inexactas o alucinaciones del modelo	El modelo genera informacion incorrecta o inventada sobre contenido universitario	Alta	Alto	Alto	AI/RAG Engineer (Diego)
R03	Problemas de integracion API entre Moodle y RAG	Errores en la comunicacion entre el modulo chat de Moodle y el sistema RAG	Media	Alto	Alto	Frontend (Percy) / Backend (Lorena)
R04	Latencia elevada en tiempos de respuesta	Los usuarios esperan demasiado tiempo para recibir una respuesta del asistente	Media	Medio	Medio	AI/RAG Engineer (Diego)
R05	Vulnerabilidades de seguridad en la API	Acceso no autorizado, inyeccion de codigo u otros ataques al sistema	Baja	Alto	Medio	DevOps (Pablo) / QA Lead (José)
R06	Perdida o corrupcion de la base de conocimiento	Los documentos procesados se pierden o danan, afectando la calidad de las respuestas	Baja	Alto	Medio	AI/RAG Engineer (Diego)
R07	Fallos en el procesamiento de documentos	Errores al ingerir documentos PDF, Word u otros formatos universitarios	Media	Medio	Medio	AI/RAG Engineer (Diego)
R08	Incompatibilidad con versiones de Moodle	El modulo de chat no funciona correctamente en distintas versiones de Moodle	Baja	Medio	Bajo	Frontend (Percy)
R09	Alcance del proyecto (scope creep)	Solicitudes de funcionalidades fuera del alcance definido que retrasen el proyecto	Alta	Medio	Alto	Product Owner (Jacknylls)
R10	Limitaciones de recursos de nube (AWS)	Exceso de uso de servicios en la nube que genere costos inesperados o limites de cuota	Media	Baja	Bajo	DevOps (Pablo)
R11	Rotacion o disponibilidad del equipo	Miembros del equipo no puedan cumplir con sus responsabilidades a tiempo	Baja	Medio	Bajo	Scrum Master (José)
R12	Cambios en la API de Amazon Bedrock	Actualizaciones que modifiquen comportamientos o keys de acceso	Baja	Medio	Bajo	AI/RAG Engineer (Diego)
R13	Problemas de rendimiento con documentos grandes	Lentitud o fallos al procesar documentos de alto volumen o peso	Media	Medio	Medio	DevOps (Pablo) / AI/RAG Engineer (Diego)
R14	Falta de datos de prueba suficientes	No contar con documentos universitarios reales para probar el sistema RAG	Media	Medio	Medio	Product Owner (Jacknylls) / QA Lead (José)

Nivel de riesgo inherente = Probabilidad x Impacto (Bajo 1-2, Medio 3-4, Alto 6-9). 14 riesgos identificados.

2. CRITERIOS DE EVALUACION Y MATRICES

2.1 Criterios de evaluacion (Probabilidad e Impacto)

Nivel	Probabilidad	Impacto
Bajo (1)	Poco probable: se estima que el evento no ocurre en la mayoria de los escenarios (baja exposicion del proyecto a la causa).	Menor: perdida o retraso de poco valor; no impide cumplir el objetivo del entregable.
Medio (2)	Posible: se estima que el evento puede ocurrir ocasionalmente; depende de factores tecnicos o externos sujetos a cambio.	Significativo: retraso notable o afectacion parcial de la funcionalidad; requiere esfuerzo de correccion.
Alto (3)	Frecuente: se estima que el evento es esperable sin una gestion activa; la causa es parte intrinseca del contexto.	Critico: perdida grave de funcionalidad, datos o continuidad del servicio; compromete la entrega del proyecto.

2.2 Matriz de nivel de riesgo inherente (Probabilidad x Impacto)

Probabilidad \ Impacto	Bajo (1)	Medio (2)	Alto (3)
Alta (3)	**3 (Medio)**	**6 (Alto)**	**9 (Alto)**
Media (2)	**2 (Bajo)**	**4 (Medio)**	**6 (Alto)**
Baja (1)	**1 (Bajo)**	**2 (Bajo)**	**3 (Medio)**

- Nivel Bajo (1-2): riesgo aceptable; se documenta y monitorea.
- Nivel Medio (3-4): riesgo tolerable; requiere acciones de mitigacion.
- Nivel Alto (6-9): riesgo critico; requiere mitigacion prioritaria y seguimiento continuo.

2.3 Matriz de riesgos inherentes (Ubicacion de los riesgos)

	Impacto Bajo	Impacto Medio	Impacto Alto
Probabilidad Alta		R09	R02
Probabilidad Media	R10	R04, R07, R13, R14	R01, R03
Probabilidad Baja		R08, R11, R12	R05, R06

3. PLAN DE RESPUESTA A RIESGOS (MITIGACIÓN)				
ID	**Riesgo**	**Estrategia**	**Responsable**	**Acciones de mitigación**
R01	Fallo en el servicio de IA (Amazon Bedrock)	Mitigación	DevOps (Pablo)	Reintentos con backoff exponencial, alarmas en CloudWatch y fallback ante caída de Bedrock; validado con prueba de inyección de errores.
R02	Respuestas inexactas o alucinaciones del modelo	Mitigación	AI/RAG Engineer (Diego)	Temperatura baja y umbral de relevancia >= 0.7 en RAG, pruebas de verificación (>= 90% de aciertos) y feedback de usuarios en el chat.
R03	Problemas de integracion API entre Moodle y RAG	Mitigación	Frontend (Percy) / Backend (Lorena)	Contrato de API congelado, pruebas de integración automatizadas y prueba E2E conjunta Moodle - API - RAG en cada sprint.
R04	Latencia elevada en tiempos de respuesta	Mitigación	AI/RAG Engineer (Diego)	Pruebas de rendimiento con JMeter (objetivo <= 5 s), optimización del pipeline RAG y caché en memoria para consultas frecuentes.
R05	Vulnerabilidades de seguridad en la API	Mitigación	DevOps (Pablo) / QA Lead (José)	Validación de seguridad con OWASP API Security Top 10, autenticación por roles, sanitización de entradas y pentest con OWASP ZAP.
R06	Perdida o corrupcion de la base de conocimiento	Mitigación	AI/RAG Engineer (Diego)	Backups diarios en S3 con versionado, verificación de integridad (hash) y versionado de la base de conocimiento para restauración.
R07	Fallos en el procesamiento de documentos	Mitigación	AI/RAG Engineer (Diego)	Validación de tipo de archivo y codificación, manejo de excepciones con logs y prueba de ingesta con al menos 10 documentos.
R08	Incompatibilidad con versiones de Moodle	Mitigación	Frontend (Percy)	Versión mínima de Moodle definida y probada (objetivo y anterior), con documentación de compatibilidad en el repositorio.
R09	Alcance del proyecto (scope creep)	Mitigación	Product Owner (Jack)	Registro y evaluación de solicitudes en el backlog, aprobación de cambios de alcance en Sprint Review y revisión por sprint.
R10	Limitaciones de recursos de nube (AWS)	Mitigación	DevOps (Pablo)	Alertas de facturación AWS (80%/100%), límite de consumo por consulta y monitoreo de tokens y costos en las pruebas del Sprint 3.
R11	Rotacion o disponibilidad del equipo	Mitigación	Scrum Master (José)	Documentación de tareas y configuraciones, revisor secundario por módulo crítico y seguimiento de bloqueos en reuniones Scrum.
R12	Cambios en la API de Amazon Bedrock	Mitigación	AI/RAG Engineer (Diego)	Revisión de notificaciones y cambios de AWS Bedrock por sprint, integración modular y pruebas de contrato en CI.
R13	Problemas de rendimiento con documentos grandes	Mitigación	DevOps (Pablo) / AI/RAG Engineer (Diego)	Límite de tamaño por documento (max. 10 MB) con rechazo, procesamiento por lotes y prueba de rendimiento con documentos grandes.
R14	Falta de datos de prueba suficientes	Mitigación	Product Owner (Jack) / QA Lead (José)	Obtención de documentos universitarios reales en el Sprint 1, generación de PDFs de prueba y base mínima de 15 documentos validada.

3.4. Plan de pruebas

						PLAN DE PRUEBAS DE SOFTWARE			Version: 1.0 Código: FPR01 Fecha de Inicio: 17/09/2026	
Item	**Grupo de pruebas**	**Tipo de Pruebas**	**Módulo**	**Id HU**	**ID CP**	**Descripción de Caso de Prueba**	**Rol**	**Estado**	**Analsita de calidad**	**Herramientas de prueba**
1	Funcionales	Funcionales	Chat RAG (Consulta)	HU-01	CP-001	Validar que el usuario realiza una consulta universitaria y recibe una respuesta correcta, relevante y contextualizada según la base de conocimiento.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Moodle / Postman / AWS Bedrock
2	Funcionales	Funcionales	Recuperación de información	HU-02	CP-002	Validar que el RAG recupera información relevante y que la respuesta incluye la fuente utilizada.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Postman / AWS Bedrock
3	Funcionales	Funcionales	Recuperación de información	HU-02	CP-003	Validar que el asistente indique cuando no dispone de información, evitando respuestas inventadas.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Postman / AWS Bedrock
4	Funcionales	Funcionales	Base de conocimiento	HU-03	CP-004	Validar la carga, actualización y eliminación de documentos y su reflejo en las respuestas.	Administrador	PENDIENTE	José Medina	Postman / Python
5	Funcionales	Funcionales	Chat RAG (Consulta)	HU-04	CP-005	Validar que el historial de conversación se mantenga y que las preguntas de seguimiento consideren el contexto previo.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Moodle / Postman
6	Técnicas	Integración	API RAG - Moodle	HU-06	CP-006	Validar el flujo completo desde Moodle hasta el procesamiento RAG y el retorno de la respuesta.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Moodle / Postman / AWS CloudWatch
7	Técnicas	API	API RAG	HU-06	CP-007	Validar los endpoints, códigos HTTP, parámetros de entrada y mensajes de error de la API.	QA	PENDIENTE	José Medina	Postman / pytest
8	Técnicas	Base de datos	Base de conocimiento / Índice vectorial	HU-03	CP-008	Validar la integridad y consistencia de los embeddings y del índice vectorial tras procesar documentos.	Administrador	PENDIENTE	José Medina	Python / AWS Bedrock / Base vectorial
9	Técnicas	Seguridad	API RAG	HU-06	CP-009	Validar la autenticación, control de acceso y permisos AWS IAM para el uso de Bedrock.	QA	PENDIENTE	José Medina	Postman / AWS IAM
10	Técnicas	Seguridad	API RAG	HU-06	CP-010	Validar la prevención de prompt injection, saneamiento de entradas y protección de información sensible.	QA	PENDIENTE	José Medina	OWASP ZAP / Postman
11	No funcionales	Rendimiento	API RAG	HU-06	CP-011	Validar que el tiempo de respuesta de la API sea ≤ 5 segundos en condiciones normales.	QA	PENDIENTE	José Medina	Jmeter
12	No funcionales	Carga	API RAG	HU-06	CP-012	Validar el comportamiento del sistema ante múltiples consultas simultáneas sin degradación significativa.	QA	PENDIENTE	José Medina	JMeter
13	No funcionales	Usabilidad	Chat RAG	HU-01	CP-013	Validar que las respuestas sean claras, legibles y comprensibles, incluyendo el manejo de errores.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Moodle / Formulario de evaluación
14	No funcionales	Compatibilidad	Chat RAG	HU-01	CP-014	Validar el funcionamiento del módulo de chat en los navegadores y dispositivos soportados por Moodle.	Estudiante / Docente / Administración	PENDIENTE	José Medina	Moodle / Navegadores
15	Regresión	Regresión	Chat RAG	HU-01 / HU-02	CP-015	Validar que las consultas previamente probadas continúen generando resultados correctos después de cambios en el RAG.	QA	PENDIENTE	José Medina	Postman / pytest

3.5. Historias de usuario

Historia de usuario HU-01	
Código	HU-01
Usuario	Estudiante o docente
Nombre historia	Realizar consultas universitarias en lenguaje natural desde el chat de Moodle
Prioridad en negocio	Alto
Riesgo en desarrollo	Alto
Programador responsable	Percy Alonzo Chabria Loayza - Frontend / Moodle
Descripción	Como estudiante o docente, quiero escribir una consulta universitaria en lenguaje natural desde el módulo de chat de Moodle, para obtener una respuesta relevante y contextualizada sin utilizar comandos especiales. La consulta debe ser enviada al sistema RAG, procesada y devuelta al chat para su visualización.
Criterios de aceptación	- El usuario puede escribir y enviar una consulta desde el chat de Moodle. - La consulta llega al sistema RAG y se procesa mediante la API REST. - El sistema genera una respuesta basada en el contexto recuperado y la muestra en el chat. - El 95 % de las consultas recibe respuesta en un máximo de 10 segundos bajo la carga definida. - La respuesta se presenta de forma clara y legible.

Historia de usuario HU-02	
Código	HU-02
Usuario	Estudiante o docente

Nombre historia	Recuperar información relevante y mostrar las fuentes utilizadas
Prioridad en negocio	Alto
Riesgo en desarrollo	Alto
Programador responsable	Diego Fernando Napa Alvarez - AI / RAG Engineer
Descripción	Como estudiante o docente, quiero que el sistema recupere los fragmentos más relevantes de la base de conocimiento y muestre las fuentes utilizadas, para confiar en que la respuesta se basa en información universitaria disponible.
Criterios de aceptación	- El sistema recupera fragmentos relacionados con el contenido de la consulta. - La respuesta incluye la fuente o referencia del documento utilizado. - La respuesta se genera con el contexto recuperado. - Cuando la base de conocimiento no contiene información suficiente, el sistema lo informa y evita inventar una respuesta. - Al menos el 80 % de las consultas del conjunto de evaluación obtiene una respuesta relevante.

Historia de usuario HU-03	
Código	HU-03
Usuario	Administrador de la base de conocimiento
Nombre historia	Gestionar los documentos y el índice vectorial de la base de conocimiento
Prioridad en negocio	Alto
Riesgo en desarrollo	Medio
Programador responsable	Diego Fernando Napa Alvarez - AI / RAG Engineer
Descripción	Como administrador de la base de conocimiento, quiero cargar, actualizar y eliminar documentos

	universitarios, para mantener vigente la información que el sistema utiliza en sus respuestas. El procesamiento debe preparar el contenido, generar embeddings y mantener actualizado el índice vectorial.
Criterios de aceptación	- El administrador puede cargar documentos definidos como fuente de conocimiento. - El sistema procesa el contenido, lo divide en fragmentos y genera sus embeddings. - Los embeddings quedan almacenados y disponibles para la búsqueda semántica. - La actualización o eliminación de un documento se refleja en las consultas posteriores. - El índice vectorial conserva la integridad y consistencia de los datos procesados.

Historia de usuario HU-04	
Código	HU-04
Usuario	Estudiante o docente
Nombre historia	Mantener el historial y el contexto de la conversación
Prioridad en negocio	Medio
Riesgo en desarrollo	Medio
Programador responsable	Percy Alonzo Chabria Loayza - Frontend / Moodle
Descripción	Como estudiante o docente, quiero conservar el historial de la conversación y realizar preguntas de seguimiento, para continuar la consulta sin repetir el contexto de las preguntas anteriores.
Criterios de aceptación	- El chat conserva las preguntas y respuestas de la sesión. - Una pregunta de seguimiento puede utilizar el contexto previo de la conversación. - El historial se muestra de forma ordenada y legible dentro del chat. - Un usuario puede realizar una consulta sin necesitar instrucciones técnicas adicionales.

Historia de usuario HU-05	
Código	HU-05
Usuario	Estudiante, docente o administrador
Nombre historia	Recibir mensajes controlados cuando la información sea insuficiente o ocurra un error
Prioridad en negocio	Alto
Riesgo en desarrollo	Alto
Programador responsable	Lorena Kim Negrillo - Backend / API Developer
Descripción	Como usuario del asistente, quiero recibir un mensaje claro cuando no exista información suficiente o se produzca un error de comunicación, para comprender la situación y continuar utilizando el sistema sin recibir una respuesta incompleta o inventada.
Criterios de aceptación	- Ante información insuficiente, el sistema comunica que no encontró contenido suficiente para responder. - Ante una falla entre Moodle, la API o el sistema RAG, se muestra un mensaje comprensible. - Los errores generan respuestas controladas sin interrumpir completamente el servicio. - Los eventos registran fecha, hora, estado y tiempo de respuesta sin almacenar información sensible. - Las solicitudes inválidas o no autorizadas reciben una respuesta controlada.

Historia de usuario HU-06	
Código	HU-06
Usuario	Estudiante o docente mediante Moodle y sistema RAG
Nombre historia	Integrar el chat de Moodle con la API REST del sistema RAG de forma segura

Prioridad en negocio	Alto
Riesgo en desarrollo	Alto
Programador responsable	Lorena Kim Negrillo - Backend / API Developer y Percy Alonzo Chabria Loayza - Frontend / Moodle
Descripción	Como responsable de la integración, quiero que Moodle y el sistema RAG se comuniquen mediante una API REST segura y consistente, para que las consultas viajen correctamente, se validen antes de procesarse y las respuestas regresen al chat.
Criterios de aceptación	- Moodle envía la consulta al endpoint de la API y recibe la respuesta generada. - La API intercambia información en formato JSON mediante HTTPS. - Las solicitudes inválidas o no autorizadas son rechazadas. - La comunicación protege los datos sensibles y las credenciales de los servicios. - El sistema soporta al menos 10 consultas simultáneas durante las pruebas de carga. - La integración se mantiene operativa y compatible con el módulo de chat.

Historia de usuario HU-07	
Código	HU-07
Usuario	Analista de calidad o administrador técnico
Nombre historia	Verificar la operación, seguridad y calidad del sistema RAG
Prioridad en negocio	Medio
Riesgo en desarrollo	Medio
Programador responsable	José Daniel Medina García - Scrum Master / QA Lead
Descripción	Como analista de calidad o administrador técnico, quiero contar con registros, controles y una arquitectura mantenible, para verificar el

	funcionamiento del sistema RAG, detectar incidencias y validar que las funcionalidades cumplan los criterios de calidad.
Criterios de aceptación	- Se registran los eventos importantes de las solicitudes y respuestas sin exponer datos sensibles. - El acceso a la API y el uso de los servicios externos se validan y controlan. - El código se mantiene modular, versionado y con documentación técnica básica. - El sistema se puede validar mediante pruebas unitarias, de integración, funcionales y de rendimiento. - La solución soporta múltiples consultas simultáneas y mantiene una disponibilidad mínima de 95 % durante el periodo de pruebas.

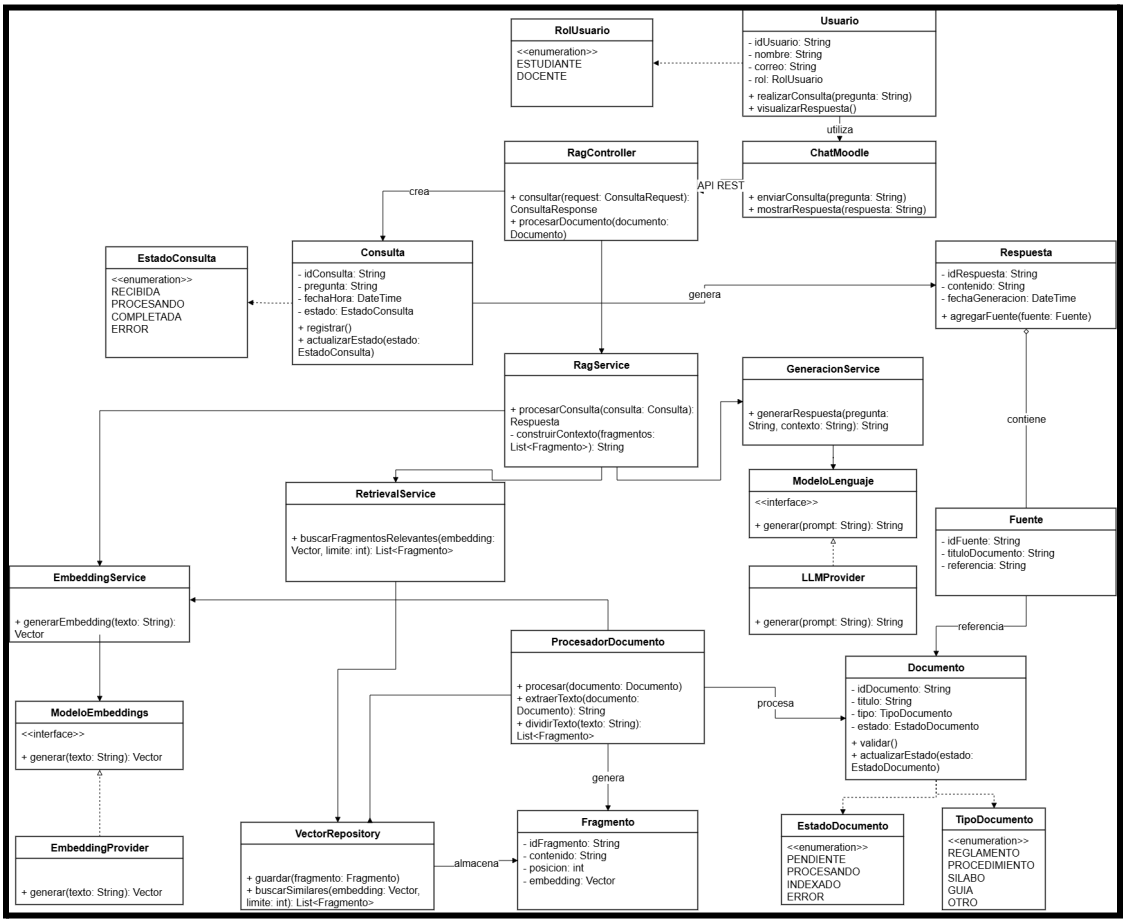
3.6. Matriz de trazabilidad HU x RF/RNF

	HU-01	HU-02	HU-03	HU-04	HU-05	HU-06	HU-07	Verificación
RF-01	x			x				Revisar
RF-02	x					x		Revisar
RF-03	x					x		Revisar
RF-04			x					Revisar
RF-05			x					Revisar
RF-06	x	x						Revisar
RF-07	x	x						Revisar
RF-08	x					x		Revisar
RF-09	x			x				Revisar
RF-10		x			x			Revisar
RF-11					x	x		Revisar
RNF-01	x					x		Revisar
RNF-02						x	x	Revisar
RNF-03						x	x	Revisar
RNF-04					x	x	x	Revisar
RNF-05	x			x				Revisar
RNF-06	x	x			x			Revisar

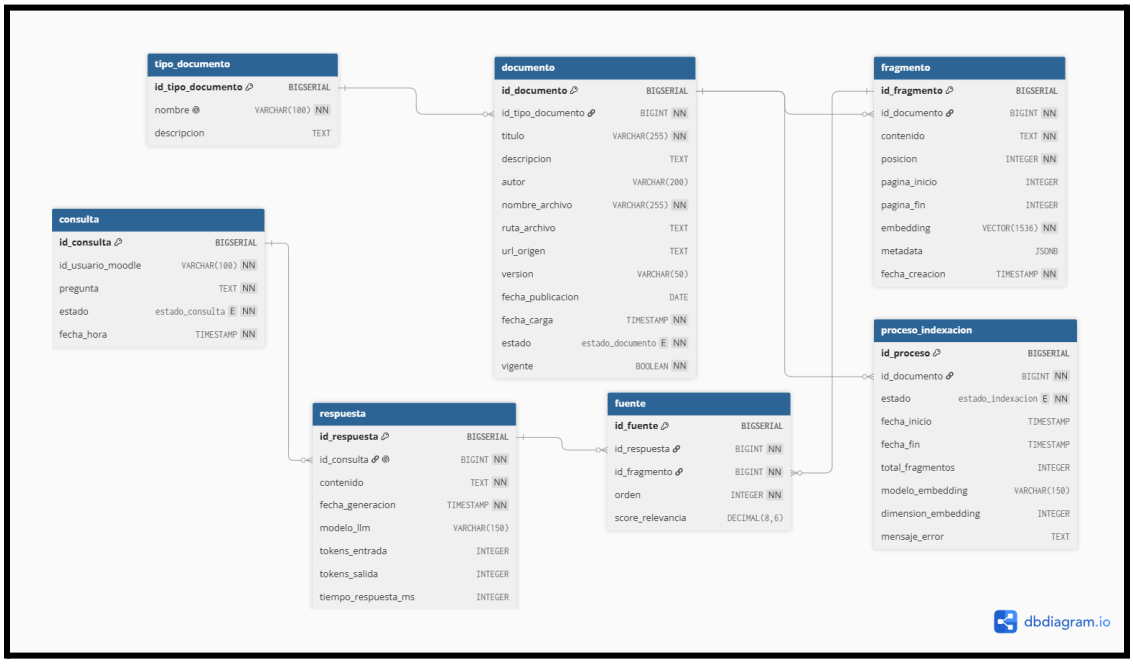
RNF-07						x	x	Revisar
RNF-08						x		Revisar
RNF-09			x			x	x	Revisar
RNF-10					x	x	x	Revisar
RNF-11					x	x		Revisar
RNF-12							x	Revisar
Verificación	Revisar	Revisar	Revisar	Revisar	Revisar	Revisar	Revisar	

4. Diseño

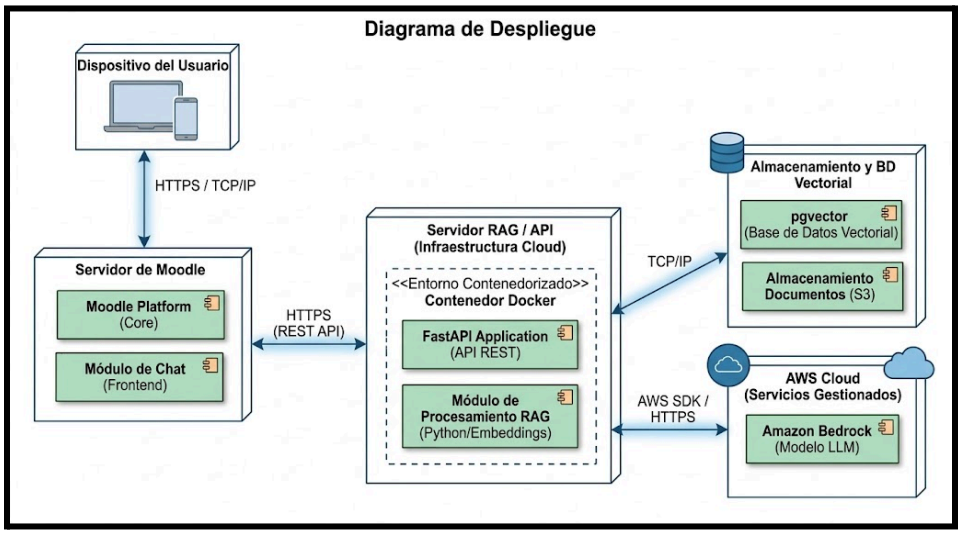
4.1. Diagrama de clases UML



4.2. Modelo Entidad Relación

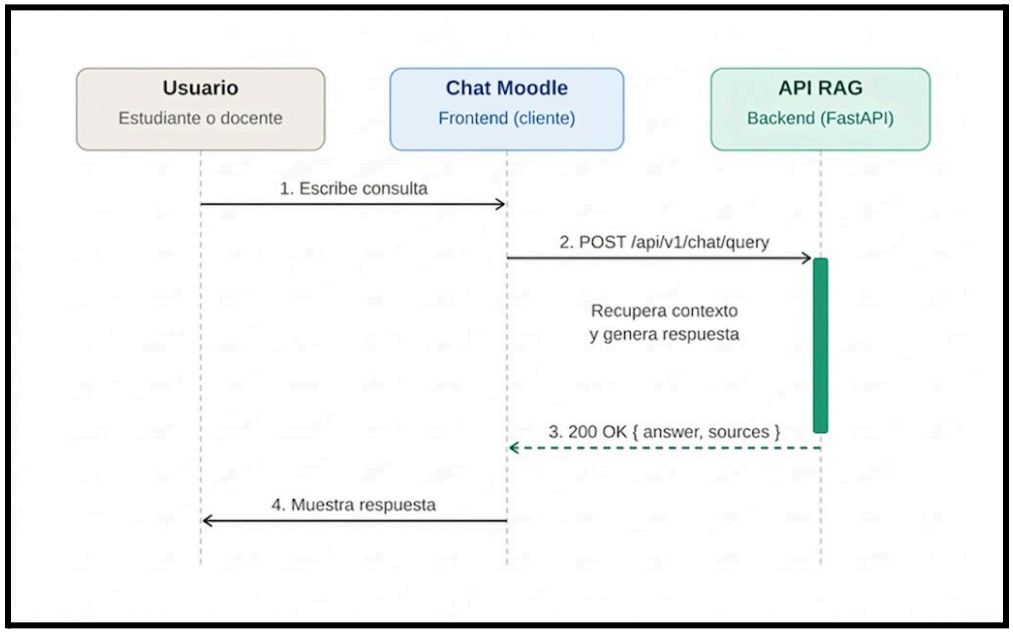


4.3. Diagrama de despliegue

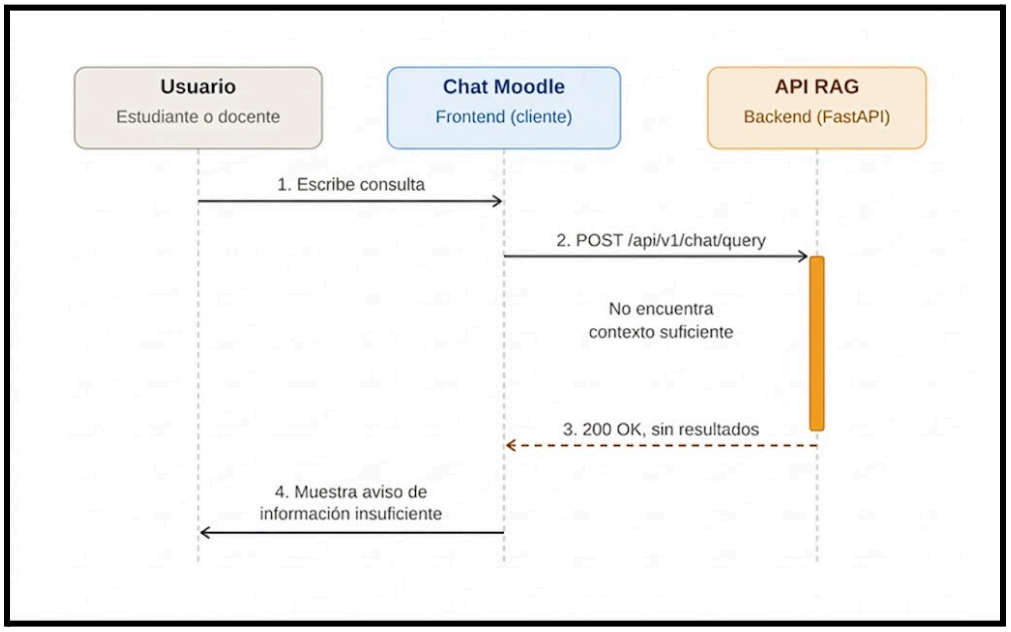


4.4. Diagramas de secuencia

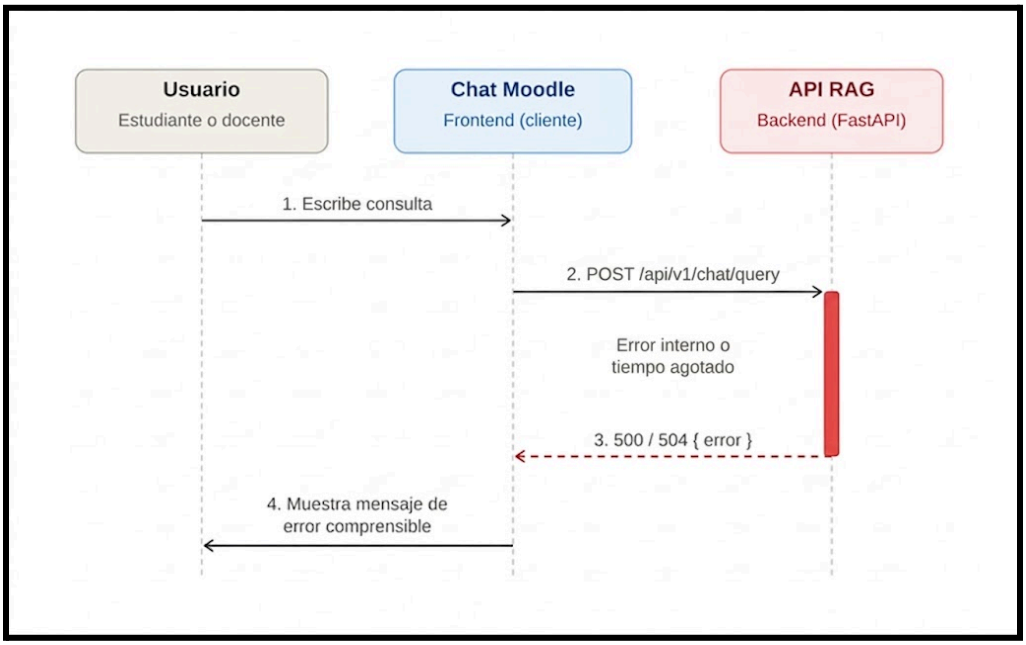
4.4.1. Diagrama - Flujo principal (consulta exitosa)



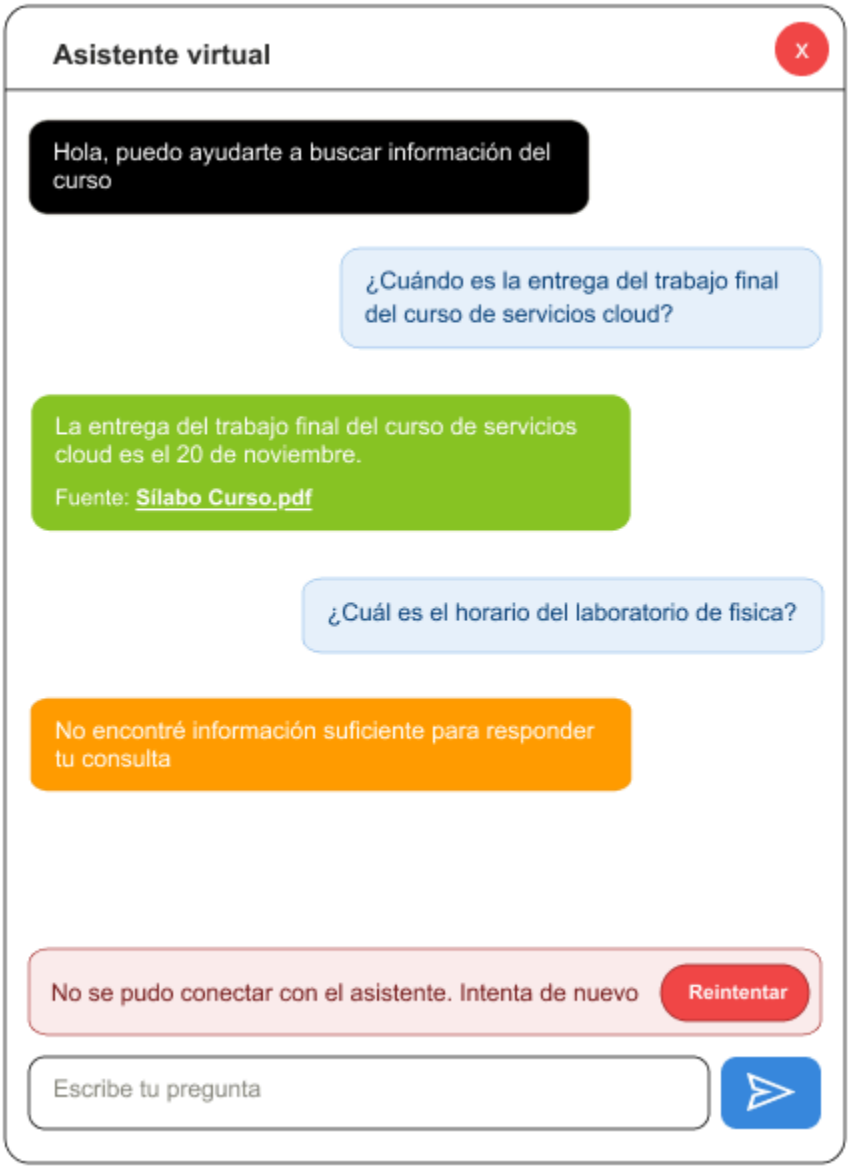
4.4.2. Diagrama - Flujo alterno (información insuficiente)



4.4.3. Diagrama - Flujo alterno (error de comunicación)



4.5. Mockup inicial



5. Conclusiones

- Se definió una solución orientada a resolver la dificultad de acceder rápidamente a información universitaria, mediante un sistema RAG que procesa documentos, recupera información relevante y genera respuestas contextualizadas.

- La separación entre Moodle, la API REST y el sistema RAG permite establecer responsabilidades claras entre los componentes, facilitando la integración del asistente sin modificar las funcionalidades académicas propias de Moodle.
- Los requerimientos e historias de usuario permitieron definir criterios verificables para las principales funciones del sistema, como consultas en lenguaje natural, recuperación con fuentes, gestión de documentos, historial, manejo de errores e integración segura.
- La matriz de trazabilidad establece la relación entre historias de usuario y requerimientos funcionales y no funcionales, proporcionando una base para comprobar que las funcionalidades definidas puedan ser verificadas durante las pruebas.
- El diseño propuesto y el plan de pruebas permiten abordar la calidad del sistema desde diferentes perspectivas, incluyendo funcionalidad, integración, rendimiento, seguridad, usabilidad y regresión, alineadas con los objetivos planteados para la validación.

6. Referencias

Khan, U. H., Khan, M. H., & Ali, R. (2025). Large language model based educational virtual assistant using RAG framework. *Procedia Computer Science, 252*, 905–911. https://doi.org/10.1016/j.procs.2025.01.051

Kuratomi, G., Pirozelli, P., Cozman, F. G., & Peres, S. M. (2025). *A RAG-based institutional assistant*. arXiv. https://doi.org/10.48550/arXiv.2501.13880

Németh, R., Tátrai, A., Szabó, M., Zaletnyik, P. T., & Tamási, Á. (2025). Exploring the use of retrieval-augmented generation models in higher education: A pilot study on

artificial intelligence-based tutoring. *Social Sciences & Humanities Open, 12*, 101751. https://doi.org/10.1016/j.ssaho.2025.101751